Cerrajería 24 Horas

Documento guía de funcionamiento de la aplicación

Idea y propósito · Cambios realizados · Ajustes pendientes con la base de datos

Proyecto Android: com.example.cerrajeria24horas

Sitio web: cerrajeria24horas.com.co

Documento de traspaso para desarrolladores y administradores.

Contenido de este documento

- 1. Idea y propósito de la aplicación.
- 2. Cómo funciona (arquitectura, pantallas, modelo de datos).
- 3. Cambios realizados hasta la fecha.
- 4. Ajustes pendientes que dependen de la base de datos (con contexto y pasos).
- 5. Cómo compilar, ejecutar y verificar.
- Anexo: historial de cambios (commits).

1. Idea y propósito de la aplicación

Cerrajería 24 Horas es una aplicación Android para la gestión diaria de un negocio de cerrajería a domicilio. Nace de una necesidad concreta: llevar el control de los clientes y de los trabajos de cerrajería (aperturas, cambios de cerradura, duplicados de llaves, aperturas de vehículos, etc.) desde el teléfono, sin depender del papel ni de hojas de cálculo.

¿Qué problema resuelve?

- Centraliza el registro de clientes y de los servicios prestados en un solo lugar.
- Permite registrar cada servicio con su cliente, tipo, dirección, fecha y valor.
- Lleva el estado de cada trabajo: Pendiente, En proceso, Realizado o Cancelado.
- Calcula las ganancias del día, la semana y el mes, con un gráfico semanal.
- Permite que un administrador (dueño del taller) cree cuentas para sus cerrajeros y les asigne los trabajos.
- Refuerza la privacidad: la app se puede bloquear con PIN y los montos se pueden ocultar.

¿Para quién es?

- Administrador (dueño del taller): ve los clientes, todos los servicios, las ganancias y su equipo. Es quien crea las cuentas de los cerrajeros.
- Cerrajero: accede con la cuenta que le creó su administrador y solo ve los servicios que tiene asignados.

Alcance del proyecto

- Aplicación Android nativa (Kotlin) como producto principal.
- Sitio web público (carpeta web/) con la información del negocio y un formulario que envía la solicitud de servicio por WhatsApp.
- Datos en la nube con Firebase (Authentication para las cuentas y Firestore para los datos).

2. Cómo funciona

2.1 Arquitectura técnica

- Lenguaje: Kotlin, con interfaz basada en XML (Views) y Material Design.
- Backend: Firebase. Authentication para el registro y el acceso; Cloud Firestore como base de datos de documentos (usuarios, clientes, servicios e historial).
- minSdk 24 (Android 7.0) y targetSdk/compileSdk 37. versionName 1.5 (versionCode 6).
- Librerías principales: Firebase BOM 34.18.0 (Firestore + Auth), Material Components, ViewPager2, DynamicAnimation y Core SplashScreen. No se usan librerías de gráficos ni de imágenes de terceros.
- El archivo app/google-services.json conecta la app con el proyecto de Firebase.

2.2 Pantallas y flujo principal

- Bienvenida (BienvenidaActivity): pantalla de splash y portada con un carrusel automático de servicios, el logotipo, acceso a Iniciar sesión / Crear cuenta y un enlace a la página web. Se muestra siempre al abrir la app.
- Login y Registro (LoginActivity, RegisterActivity): acceso con correo y contraseña; registro de la cuenta del administrador; recuperación de contraseña.
- Panel principal (MainActivity): contiene cinco secciones en una barra inferior flotante y un botón flotante central para crear un servicio:
- Inicio: resumen con saludo, indicadores del día y del mes, gráfico de ganancias y un anillo de progreso.
- Servicios: lista de trabajos con filtros por estado, agrupación y acceso al detalle con línea de tiempo de cambios de estado.
- Clientes: lista de clientes con foto/avatar, búsqueda y acceso al detalle y a la edición.
- Ganancias: montos del día, la semana y el mes, más un gráfico de barras semanal.
- Equipo: (solo administrador) lista del taller y creación de cuentas de cerrajero.
- Formularios y detalle (ClientFormActivity, ServicioFormActivity, ClienteDetalleActivity, ServicioDetalleActivity): alta y edición de clientes y servicios, y generación de la factura en PDF.
- Perfil, Ajustes, Bloqueo y Legal (PerfilActivity, AjustesActivity, BloqueoActivity, LegalActivity): foto de perfil, control de sesión, PIN de bloqueo, reducir movimiento y textos legales (política de privacidad y términos).

2.3 Modelo de datos (Cloud Firestore)

Firestore organiza la información en colecciones de documentos. Las colecciones que usa la app son las siguientes (definidas en data/FirestoreAPI.kt y data/Modelos.kt).

Colección	Campo	Descripción
usuarios	uid	Identificador del usuario (igual al UID de Firebase

		Auth).
	nombre, correo, telefono	Datos de la cuenta.
	rol	'admin' o 'cerrajero'.
	tallerId	Identifica el taller al que pertenece. Para el administrador es su propio uid.
	fechaCreacion	Fecha de creación de la cuenta.
clientes	clienteId	Identificador del cliente (igual al id del documento).
	nombreCompleto, telefono, correo	Datos de contacto.
	ciudad, direccion, notas	Ubicación y observaciones.
	fechaRegistro	Fecha de alta del cliente.
	registradoPor	uid del usuario que registró al cliente.
servicios	servicioId	Identificador del servicio (igual al id del documento).
	clienteId, nombreCliente	Cliente asociado al servicio.
	tipoServicio	Tipo de trabajo (ver lista de tipos más abajo).
	descripcion, ciudad, direccion	Detalle y ubicación del trabajo.
	fecha	Fecha y hora programada del servicio.
	valor	Valor cobrado (número).
	estado	PENDIENTE, EN_PROCESO, REALIZADO o CANCELADO.
	fechaCreacion, fechaFinalizacion	Alta y cierre (se llena al pasar a Realizado).
	cerrajeroId	uid del usuario que creó/registró el servicio.
	asignadoA	uid del cerrajero al que se le asignó el servicio (vacío = sin asignar).
historialEstados	historialId, servicioId	Registro del cambio de

		estado de un servicio.
	estadoAnterior, estadoNuevo	Transición de estado.
	fecha, usuarioId	Cuándo y quién hizo el cambio.

Estados y tipos de servicio

Estados posibles de un servicio (EstadoServicio):

- PENDIENTE: registrado, sin iniciar.
- EN_PROCESO: en ejecución.
- REALIZADO: terminado (guarda la fecha de finalización).
- CANCELADO: anulado.

Tipos de servicio (TiposServicio):

- APERTURA_DE_PUERTA, CAMBIO_DE_CERRADURA, INSTALACION_DE_CERRADURA, REPARACION_DE_CERRADURA, DUPLICADO_DE_LLAVES, APERTURA_DE_VEHICULO, OTRO.

2.4 Privacidad y ajustes del dispositivo

- Bloqueo por PIN: se guarda como hash SHA-256 (nunca en texto plano).
- Ocultar montos: las cifras de dinero se pueden esconder con un toque.
- Control de sesión: cierre por inactividad configurable (Nunca, 5, 15 o 30 minutos).
- Reducir movimiento: respeta la preferencia de accesibilidad y limita las animaciones.
- Factura en PDF: se genera con FileProvider para compartirla sin salir de la app.
- Legal: política de privacidad y términos redactados dentro de la misma app.

3. Cambios realizados

A continuación se resume el trabajo realizado. El detalle cronológico completo está en el Anexo (historial de commits).

3.1 Rediseño visual e identidad

- Letrero de marca en una sola línea, tanto en el panel como en el acceso.
- Acceso (login) compacto y sin desplazamiento, con las cuentas del dispositivo en fila y cabecera que se encoge; enlace de recuperación de contraseña centrado.
- Formato de precio mientras se escribe, con un vigía (watcher) reutilizable y sus pruebas.
- Etiquetas de la barra inferior que nunca se cortan y se adaptan al ancho.
- Superficies planas, control de sesión segmentado y animaciones del PIN.
- Paleta y tipografía unificadas (familia DM Sans) y botones estandarizados; el logotipo se mantuvo intacto.

3.2 Navegación y panel

- Host de fragmentos con barra de navegación inferior flotante.
- Panel inicial tipo 'bento' (Inicio) con indicadores y accesos rápidos.
- Botón flotante central para crear un nuevo servicio.
- Carrusel de bienvenida automático (avanza solo) y sin recortes.
- Splash y onboarding de bienvenida; arranque en accesos.

3.3 Servicios, clientes y ganancias

- Servicios con chips de filtro, agrupación y detalle con línea de tiempo de estados.
- Inicio con anillo de progreso, gráfico de ganancias y avatar de clientes.
- Ganancias del día, la semana y el mes con gráfico de barras semanal.

3.4 Modelo de equipo (administrador y cerrajeros)

- Se agregaron los campos de equipo al modelo: rol, tallerId y asignadoA.
- Pantalla 'Equipo' como quinto destino, con creación de cuentas de cerrajero.
- Asignación de un servicio a un cerrajero desde el detalle, con vistas filtradas por rol.
- Las cuentas de cerrajero se crean con una instancia secundaria de Firebase para no cerrar la sesión del administrador.
- Script de migración de datos para habilitar el modelo de equipo (ver Anexo y sección 4).

3.5 Sitio web (carpeta web/)

- Landing pública con los datos reales del negocio, servicios en cuadrícula y navegación con scrollspy.
- Sección de trabajos con fotos reales de cerrajería y cuadrícula simétrica (sin huecos).
- Formulario de solicitud de servicio que envía los datos por WhatsApp.
- Fecha, hora y tipo de servicio con controles propios (calendario, selector de hora y desplegable) con el estilo de la página, en lugar de los controles nativos del navegador.

3.6 Correcciones de errores

- Se evitó el cierre inesperado al cambiar entre las vistas Inicio y Servicios (guarda en los callbacks de Firestore).
- Se ajustaron los espacios del acceso (login).
- Se quitaron los avisos de bienvenida y de cuenta creada, que resultaban molestos.
- Se alineó el botón de agregar cerrajero con el resto de secciones.
- Se corrigió una imagen del sitio web que no cargaba.

4. Ajustes pendientes (dependen de la base de datos)

Durante el rediseño se trabajó la interfaz sin modificar la capa de datos. Por eso quedan pendientes algunos ajustes que sí requieren tocar Firestore o la configuración de Firebase. Se describen con su contexto para que cualquier persona que reciba el proyecto entienda qué hacer y por qué.

#	Pendiente	Impacto
A	Datos compartidos por taller (agregar tallerId a clientes y servicios)	Alto
B	Ejecutar la migración de datos en Firestore	Alto
C	Reglas de seguridad de Firestore	Alto
D	Unificar el criterio de asignación (cerrajeroId vs asignadoA)	Medio
E	Índices compuestos de Firestore (solo si se ordena en la consulta)	Bajo
F	Revisión de configuración de Firebase y versionado antes de publicar	Medio

A. Datos compartidos por taller (lo más importante)

Contexto:

- Hoy cada cliente y cada servicio queda atado al uid de quien lo creó (campos registradoPor y cerrajeroId).
- El administrador consulta con obtenerClientes(uid) y obtenerServicios(uid), que filtran por esos campos. Por eso el administrador solo ve lo que él mismo registró y NO ve los clientes ni los servicios que crearon sus cerrajeros.
- Los modelos Cliente y Servicio aún no tienen el campo tallerId.

Qué hay que hacer:

- Agregar el campo tallerId a los modelos Cliente y Servicio (data/Modelos.kt) y a sus funciones toMap/fromSnapshot.
- Al guardar un cliente o un servicio (ClienteFormActivity, ServicioFormActivity), escribir tallerId con el taller del usuario (el tallerId de su perfil en usuarios/{uid}).

- Cambiar las consultas de la capa de datos (data/FirestoreAPI.kt) para que el administrador filtre por tallerId en lugar de por uid. Decidir si el cerrajero sigue viendo solo lo asignado (asignadoA) o también todo su taller.
- Actualizar la función obtenerServiciosSegunRol, que hoy decide entre administrador y cerrajero según el rol del perfil.

B. Ejecutar la migración de datos

Contexto:

- En firebase-migracion/ hay un script (migrar_equipo.js) que habilita el modelo de equipo en los datos que ya existían: asigna tallerId (el propio uid) y rol 'admin' a los usuarios que no los tengan, e inicializa asignadoA en '' en los servicios.
- Por defecto corre en modo simulacro: no escribe nada; solo muestra en consola los documentos que cambiaría.

Pasos:

- Descargar una cuenta de servicio desde Firebase Console (Configuración del proyecto > Cuentas de servicio > Generar nueva clave privada).
- Definir la variable de entorno GOOGLE_APPLICATION_CREDENTIALS apuntando al archivo JSON.
- Ejecutar 'npm install' dentro de firebase-migracion.
- Ejecutar 'npm run migrar' (simulación) y revisar la lista que se imprime.
- Si todo está correcto, ejecutar 'npm run migrar:aplicar' para escribir los cambios.

Importante: si se implementa el punto A, hay que extender este script para que también rellene tallerId en clientes y servicios (resolviendo el taller a partir de registradoPor y cerrajeroId).

C. Reglas de seguridad de Firestore

Contexto:

- El repositorio no incluye un archivo de reglas (firestore.rules). Sin reglas, cualquier usuario autenticado podría leer o escribir datos de otros talleres.

Qué hay que hacer:

- Crear reglas que permitan a cada usuario acceder solo a los documentos de su taller (por ejemplo, comparando el campo tallerId del documento con el tallerId del perfil en usuarios/{uid}).
- Considerar cómo se crean las cuentas: el administrador crea el documento usuarios/{uid} del cerrajero desde la app. Las reglas deben permitir esa escritura o moverla a una Cloud Function con el SDK de administrador.
- Desplegar las reglas con Firebase CLI.

D. Unificar el criterio de asignación

Contexto:

- El administrador ve servicios por cerrajeroId (quien lo creó) y el cerrajero ve los servicios por asignadoA (a quién se le asignó). La función asignarServicio solo actualiza asignadoA, así que el registro puede quedar incoherente entre las dos vistas.

Qué hay que hacer:

- Definir una única fuente de verdad: por ejemplo, que 'cerrajeroId' sea el responsable actual y que la asignación lo actualice, o migrar todas las consultas a tallerId + asignadoA.
- Ajustar en consecuencia data/Modelos.kt, data/FirestoreAPI.kt y las vistas por rol.

E. Índices compuestos (opcional)

- Hoy la app ordena los servicios en el propio dispositivo, así que no hacen falta índices.
- Si más adelante se filtra por tallerId y se ordena por fecha en la consulta, Firestore pedirá un índice compuesto que se crea desde la consola de Firebase.

F. Revisión antes de publicar

- Confirmar que app/google-services.json corresponde al proyecto de Firebase de producción (y no a uno de pruebas).
- Revisar versionCode y versionName antes de subir a Google Play.
- Probar el flujo completo con dos cuentas reales (un administrador y un cerrajero) una vez aplicados los puntos A a D.

5. Cómo compilar, ejecutar y verificar

Requisitos

- Android Studio (con el SDK de Android instalado) o el Gradle wrapper incluido.
- JDK 11 o superior (el proyecto usa Java 11 como objetivo).
- El archivo app/google-services.json del proyecto de Firebase.

Compilar y ejecutar las comprobaciones

Desde la raíz del proyecto, en PowerShell:

```
.\gradlew.bat assembleDebug testDebugUnitTest lintDebug --console=plain
```

El resultado correcto termina con 'BUILD SUCCESSFUL'. Si aparece 'BUILD FAILED' o 'FAILED', revisar el detalle en la salida.

Abrir en Android Studio

- Abrir la carpeta del proyecto con Android Studio.
- Esperar a que Gradle sincronice (descargará las dependencias).
- Ejecutar en un emulador o en un teléfono con Android 7.0 o superior.

Publicar la web

- La carpeta web/ contiene index.html, styles.css y script.js, además de assets/ (logo y favicon).
- Se puede publicar tal cual en cualquier servidor de archivos estáticos o hosting.

Nota de traspaso: las secciones 1 a 3 explican qué es la app y qué se hizo; la sección 4 es la lista de tareas que faltan y que dependen de la base de datos. Con este documento y el proyecto en mano, cualquier desarrollador puede continuar el trabajo.

Anexo: historial de cambios (commits)

Resumen de los commits más representativos, en orden cronológico (los más recientes primero). El historial completo está en el repositorio Git del proyecto.

Fase	Contenido
Rediseño (A–G)	Letrero de marca; acceso sin desplazamiento; recuperación centrada; formato de precio en vivo y pruebas; etiquetas de barra adaptables; superficies planas, sesión segmentada y animaciones de PIN.
Navegación y panel	Host de fragmentos y barra flotante; panel tipo bento; botón flotante de nuevo servicio.
Arranque y portada	Splash y onboarding; carrusel de bienvenida automático; acceso compacto.
Servicios y clientes	Chips, agrupación y detalle con línea de tiempo; inicio con anillo y gráfico; avatares de clientes.
Equipo	Quinto destino; creación de cuentas de cerrajero; asignación desde el detalle y vistas por rol; modelo de equipo y API de datos; script de migración.
Web	Landing con datos reales; servicios y trabajos en cuadrícula; fotos reales; formulario con controles propios (calendario, hora y servicio).
Correcciones	Cierre al cambiar de vista; espacios del acceso; avisos eliminados; botón de agregar cerrajero; imagen que no cargaba.

Fin del documento.